

ĐẠI HỌC QUỐC GIA TP. HCM
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN



BÁO CÁO TỔNG KẾT
ĐỀ TÀI KHOA HỌC VÀ CÔNG NGHỆ SINH VIÊN NĂM 2026

Tên đề tài tiếng Việt:

**PHÁT HIỆN BẤT THƯỜNG THỜI GIAN THỰC TRONG DỮ LIỆU TAXI
NYC SỬ DỤNG KIẾN TRÚC STREAMING ĐA LUỒNG VÀ MÔ HÌNH HỌC
MÁY KẾT HỢP**

Tên đề tài tiếng Anh:

**REAL-TIME ANOMALY DETECTION IN NYC TAXI DATA USING A
MULTI-STREAM STREAMING ARCHITECTURE AND A HYBRID
MACHINE LEARNING MODEL**

Khoa: Hệ thống thông tin

Bộ môn: Dữ Liệu Lớn

Thời gian thực hiện: 4/2026 - 6/2026

Cán bộ hướng dẫn: Nguyễn Hồ Duy Tri

Tham gia thực hiện			
TT	Họ và tên, MSSV	Chịu trách nhiệm	MSSV
1.	Phạm Hùng Quốc Việt	Nhóm trưởng	23521783
2.	Lê Ngô Minh Hoàng	Thành viên	23520523



ĐẠI HỌC QUỐC GIA TP. HCM
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN

Ngày nhận	
Mã số đề tài	
(Do CQ quản lý ghi)	

BÁO CÁO TỔNG KẾT

Tên đề tài tiếng Việt:

**PHÁT HIỆN BẤT THƯỜNG THỜI GIAN THỰC TRONG DỮ LIỆU TAXI
NYC SỬ DỤNG KIẾN TRÚC STREAMING ĐA LUỒNG VÀ MÔ HÌNH HỌC
MÁY KẾT HỢP**

Tên đề tài tiếng Anh:

**REAL-TIME ANOMALY DETECTION IN NYC TAXI DATA USING A
MULTI-STREAM STREAMING ARCHITECTURE AND A HYBRID
MACHINE LEARNING MODEL**

Ngày ... tháng năm
Cán bộ hướng dẫn
(Họ tên và chữ ký)

Ngày ... tháng năm
Sinh viên chủ nhiệm đề tài
(Họ tên và chữ ký)

THÔNG TIN KẾT QUẢ NGHIÊN CỨU

1. Thông tin chung:

- **Tên đề tài:** Phát hiện bất thường thời gian thực trong dữ liệu Taxi NYC sử dụng kiến trúc Streaming đa luồng và mô hình học máy kết hợp
- **Chủ nhiệm:** Phạm Hùng Quốc Việt (MSSV: 23521783)
- **Thành viên tham gia:** Lê Ngô Minh Hoàng (MSSV: 23520523)
- **Cơ quan chủ trì:** Trường Đại học Công nghệ Thông tin – ĐHQG TP.HCM
- **Thời gian thực hiện:** 04/2026 – 06/2026
- **Cán bộ hướng dẫn:** ThS. Nguyễn Hồ Duy Tri

2. Mục tiêu:

Xây dựng hệ thống phát hiện bất thường thời gian thực trên luồng dữ liệu taxi NYC (Yellow & Green) sử dụng kiến trúc Apache Kafka + Spark Structured Streaming + PostgreSQL. Hệ thống áp dụng mô hình học máy kết hợp (Isolation Forest + MinCovDet) với 14 đặc trưng kỹ thuật và chiến lược Strict Ensemble Voting nhằm giảm tối đa tỷ lệ cảnh báo sai trong khi vẫn duy trì khả năng phát hiện đa dạng loại gian lận với độ trễ xử lý dưới 1 giây.

3. Tính mới và sáng tạo:

- (1) Kiến trúc hai luồng Kafka song song xử lý đồng thời Yellow Taxi và Green Taxi với tốc độ trung bình ~400 rows/s tổng hợp (dao động 400–800 rows/s tùy tải batch).
- (2) Mô hình Ensemble hai tầng IF + MCD kết hợp Strict Voting — lần đầu áp dụng MinCovDet Mahalanobis Distance cho phát hiện bất thường đa biến trong dữ liệu taxi.
- (3) Cơ chế Adaptive Contamination ($0.80 \times \text{prior} + 0.20 \times \text{observed}$, clamp [0.01, 0.04]) cho phép mô hình tự điều chỉnh mà không bị Concept Drift.
- (4) Bộ 14 đặc trưng với mã hóa tuần hoàn giờ (sin/cos) và sample weighting $\times 3.0$ cho vi phạm luật nghiệp vụ.
- (5) Hệ thống chẩn đoán tự động 6 chiều (evaluate_model.py) và Benchmark Evaluator so chuẩn với Ground Truth AI offline.

4. Tóm tắt kết quả nghiên cứu:

Hệ thống đạt throughput trung bình ~400 rows/s (Yellow ~200 + Green ~200), đỉnh peak ~800 rows/s sau chu kỳ retrain, độ trễ trung bình 230 ms và tỷ lệ bất thường ổn định 1,95% trên toàn bộ 3.996.659 bản ghi. Kết quả đánh giá cho thấy Cohen's $d = 2,65$ (EXCELLENT) xác nhận mức tách biệt điểm số cao giữa chuyển đi bình thường và bất thường. Phân loại bất thường theo 9 nhãn lý do, đứng đầu là Unrealistic Speed (>140 km/h) chiếm 71% và Extreme Fare per Mile chiếm 20,8%. Business Rule Overlap đạt 33,1%, chứng minh mô hình ML bổ sung các phát hiện thống kê vượt ra ngoài luật cứng.

5. Tên sản phẩm:

- (1) Hệ thống phần mềm Real-Time Anomaly Detection Pipeline (mã nguồn mở, giấy phép MIT);
- (2) Bộ dữ liệu Ground Truth đã gán nhãn

(yellow/green_tripdata_labeled.parquet); (3) Ứng dụng web Fleet Intelligence Dashboard (Streamlit, port 8501) và Benchmark Evaluator (port 8502).

6. Hiệu quả, phương thức chuyển giao kết quả nghiên cứu và khả năng áp dụng:

Hệ thống có khả năng triển khai trực tiếp tại các công ty quản lý đội taxi, nền tảng ride-hailing hoặc cơ quan giám sát giao thông công cộng. Mã nguồn được công bố tự do trên GitHub (giấy phép MIT) kèm tài liệu vận hành đầy đủ. Kết quả nghiên cứu có thể chuyển giao dưới dạng Docker image sẵn dùng, cho phép triển khai một lần với lệnh `docker compose up`. Hệ thống có khả năng mở rộng sang các nguồn dữ liệu giao thông khác (xe buýt, xe công nghệ) bằng cách thay thế Kafka producer.

7. Hình ảnh, sơ đồ minh họa chính:

[Xem sơ đồ kiến trúc hệ thống và ảnh chụp màn hình Dashboard tại Phụ lục A]

Cơ quan Chủ trì
(ký, họ và tên, đóng dấu)

Chủ nhiệm đề tài
(ký, họ và tên)

Mục lục

THÔNG TIN KẾT QUẢ NGHIÊN CỨU.....	3
1. Thông tin chung:.....	3
2. Mục tiêu:.....	3
3. Tính mới và sáng tạo:.....	3
4. Tóm tắt kết quả nghiên cứu:.....	3
5. Tên sản phẩm:.....	3
6. Hiệu quả, phương thức chuyển giao kết quả nghiên cứu và khả năng áp dụng:...	4
7. Hình ảnh, sơ đồ minh họa chính:.....	4
Mục lục.....	5
Phục lục bảng.....	6
Phục lục hình.....	6
Tóm tắt.....	7
Abstract.....	8
I. GIỚI THIỆU.....	9
II. CƠ SỞ LÝ THUYẾT.....	9
A. Bài toán phát hiện bất thường.....	9
B. Isolation Forest.....	9
C. Minimum Covariance Determinant (MCD).....	10
D. Strict Ensemble Voting.....	10
III. KIẾN TRÚC HỆ THỐNG.....	11
A. Tổng quan kiến trúc.....	12
B. Tầng Ingestion — Hai luồng dữ liệu song song.....	12
C. Tầng Xử lý — Spark Structured Streaming + Ensemble ML.....	13
D. Cơ sở dữ liệu PostgreSQL — Schema 3 bảng.....	13
E. Tầng Visualization — Non-blocking UI.....	13
IV. KỸ THUẬT ĐẶC TRƯNG (FEATURE ENGINEERING).....	14
A. Hai giai đoạn trích xuất.....	14
B. Hệ thống phân loại lý do bất thường (9 luật nghiệp vụ).....	15
V. KHÓ KHĂN THỰC TẾ TRONG QUÁ TRÌNH TRIỂN KHAI VÀ GIẢI PHÁP.....	16
A. Thách thức 1: Throughput quá cao làm quá tải ML Inference.....	16
B. Thách thức 2: Vấn đề kiểm định kết quả — Ground Truth Problem.....	16
VI. PHƯƠNG PHÁP THỰC NGHIỆM.....	17
A. Dữ liệu thực nghiệm và môi trường thực nghiệm.....	17
B. Quy trình đánh giá 6 chiều.....	17

C. Hiệu năng hệ thống.....	18
D. Kết quả phát hiện bất thường.....	18
E. Phân loại bất thường theo loại.....	19
F. Kết quả 6 kiểm định tự động.....	19
VII. PHƯƠNG THỨC CHUYỂN GIAO KẾT QUẢ NGHIÊN CỨU VÀ KHẢ NĂNG ÁP DỤNG.....	20
A. Phương thức chuyển giao.....	20
B. Khả năng áp dụng thực tiễn.....	21
VIII. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	22
A. Kết luận.....	22
B. Hạn chế.....	22
C. Hướng phát triển.....	22
TÀI LIỆU THAM KHẢO.....	23

Phục lục bảng

Bảng	Nội dung
I	<i>Các thành phần công nghệ trong hệ thống</i>
II	<i>Bộ 14 đặc trưng ML và giải thích</i>
III	<i>Hệ thống phân loại lý do bất thường theo 9 luật cứng</i>
IV	<i>Môi trường và cấu hình hệ thống</i>
V	<i>Hiệu năng hệ thống theo luồng dữ liệu</i>
VI	<i>Kết quả phát hiện bất thường tổng hợp</i>
VII	<i>Phân loại và phân bố bất thường được phát hiện</i>
VIII	<i>Kết quả 6 kiểm định tự động (evaluate_model.py)</i>

Phục lục hình

Hình	Nội dung
1	<i>Pipeline kiến trúc streaming đa luồng và mô hình học máy</i>
2	<i>Dashboard streamlit</i>
3	<i>Dashboard streamlit</i>
4	<i>Giao diện Benchmark</i>
5	<i>Giao diện Benchmark</i>

Tóm tắt

Bài báo trình bày thiết kế và triển khai hệ thống phát hiện bất thường thời gian thực cho dữ liệu taxi thành phố New York (NYC). Kiến trúc gồm hai luồng dữ liệu song song (Yellow Taxi và Green Taxi) được đưa vào Apache Kafka, xử lý bởi Spark Structured Streaming, phân loại bằng mô hình học máy kết hợp Isolation Forest (IF) và Minimum Covariance Determinant (MCD), sau đó lưu trữ vào PostgreSQL và trực quan hóa qua Streamlit. Chiến lược ensemble voting nghiêm ngặt (Strict Mode) — chỉ gán nhãn bất thường khi cả hai mô hình đồng thuận — giúp giảm tỷ lệ cảnh báo sai trong khi duy trì khả năng phát hiện đa dạng: tốc độ bất thường, cước phí phi lý, thời gian hành trình không nhất quán và nhiều vi phạm quy tắc nghiệp vụ. Thử nghiệm trên bộ dữ liệu NYC TLC tháng 3/2026 với 3.996.659 bản ghi cho thấy hệ thống đạt throughput tổng hợp trung bình ~400 rows/s (dao động 400–800 rows/s theo từng micro-batch), độ trễ trung bình 230 ms và tỷ lệ bất thường ổn định 1,95%. Đánh giá mô hình với Cohen's $d = 2,65$ xác nhận mức tách biệt điểm số cao giữa chuyến đi bình thường và bất thường.

Từ khóa — Phát hiện bất thường, Kafka, Spark Structured Streaming, Isolation Forest, MinCovDet, Ensemble Learning, NYC Taxi, Kiến trúc dữ liệu thời gian thực.

Abstract

This paper presents the design and implementation of a real-time anomaly detection system for New York City (NYC) taxi trip data. The architecture comprises two parallel data streams (Yellow Taxi and Green Taxi) ingested via Apache Kafka, processed by Spark Structured Streaming, classified using an Isolation Forest (IF) and Minimum Covariance Determinant (MCD) ensemble, persisted in PostgreSQL, and visualized through a Streamlit dashboard. A strict ensemble voting strategy — flagging anomalies only when both models agree — substantially reduces false-positive alerts while preserving broad fraud-pattern coverage: unrealistic speeds, implausible fares, inconsistent trip durations, and various business-rule violations. Experiments on the March 2026 NYC TLC dataset (3,996,659 records) demonstrate ~400 rows/s average (peak up to 800 rows/s) combined throughput, 230 ms average latency, and a stable 1.95% anomaly rate. Model evaluation yields Cohen's $d = 2.65$, confirming strong score separation between normal and anomalous trips. Keywords — Anomaly Detection, Kafka, Spark Structured Streaming, Isolation Forest, MinCovDet, Ensemble Learning, NYC Taxi, Real-Time Data Architecture.

I. GIỚI THIỆU

Dữ liệu hành trình taxi đô thị là một trong những nguồn tài nguyên phong phú nhất để nghiên cứu hành vi giao thông, phát hiện gian lận cước phí và tối ưu hóa vận hành đội xe. Thành phố New York công khai bộ dữ liệu hàng tháng của các loại taxi qua cổng dữ liệu mở TLC (Taxi and Limousine Commission) [1], tạo điều kiện cho nhiều nghiên cứu học thuật. Tuy nhiên, phần lớn nghiên cứu hiện tại xử lý dữ liệu theo lô (batch processing) sau thu thập, dẫn đến độ trễ phát hiện từ vài giờ đến vài ngày.

Các hệ thống phát hiện gian lận thời gian thực đang ngày càng được chú trọng trong lĩnh vực giao thông thông minh. Zhang et al. [2] chứng minh kết hợp Isolation Forest với ngưỡng cảnh báo thời gian đạt độ chính xác cao hơn 15% so với phương pháp thống kê truyền thống. Các kiến trúc streaming như Kafka + Spark Structured Streaming [3] có khả năng xử lý hàng triệu sự kiện mỗi giây với độ trễ dưới 100 ms [4]. Nalla [5] nhấn mạnh sự chuyển dịch tất yếu từ batch sang near-real-time stream processing để tối ưu hóa quyết định thông minh trong doanh nghiệp.

Bài báo đề xuất hệ thống end-to-end tích hợp kiến trúc streaming đa luồng với mô hình học máy kết hợp (IF + MCD) để phát hiện bất thường thời gian thực trong dữ liệu taxi NYC. Đóng góp chính gồm: (i) kiến trúc hai luồng Kafka song song cho Yellow và Green Taxi; (ii) Strict Ensemble Voting giảm false positive; (iii) kỹ thuật feature engineering 14 đặc trưng với mã hóa tuần hoàn thời gian; (iv) hệ thống đánh giá tự động 6 chiều; (v) toàn bộ mã nguồn công bố mở (MIT) .

II. CƠ SỞ LÝ THUYẾT

A. Bài toán phát hiện bất thường

Phát hiện bất thường (anomaly detection) là bài toán tìm kiếm các mẫu dữ liệu lệch đáng kể so với hành vi kỳ vọng [12]. Trong ngữ cảnh dữ liệu taxi, bất thường có thể biểu hiện theo ba dạng: (1) Point Anomaly — một bản ghi riêng lẻ có giá trị đặc trưng cực kỳ bất thường, ví dụ xe taxi di chuyển 200 km/h; (2) Contextual Anomaly — bất thường chỉ trong ngữ cảnh cụ thể, ví dụ cước \$80 cho chuyến đi 2 phút; (3) Collective Anomaly — tập hợp bản ghi tạo mẫu bất thường dù mỗi bản ghi riêng lẻ trông bình thường.

Dữ liệu taxi NYC có đặc trưng mất cân bằng (class imbalance) cực kỳ cao: tỷ lệ bất thường thực tế chỉ khoảng 1–5% tổng số chuyến đi. Điều này loại trừ hầu hết các thuật toán học có giám sát (supervised learning) vì không có nhãn (labels) và phân phối mất cân bằng nghiêm trọng. Thuật toán không giám sát (unsupervised) như Isolation Forest [7] và MinCovDet [8] là lựa chọn phù hợp vì chúng học phân phối bình thường của dữ liệu mà không cần nhãn huấn luyện.

B. Isolation Forest

Isolation Forest là thuật toán học máy phi giám sát dựa trên nguyên lý: các điểm dữ liệu bất thường (outliers) thường ít và có giá trị thuộc tính khác biệt lớn so với các điểm bình thường, do đó chúng dễ bị cô lập hơn trong cấu trúc cây nhị phân ngẫu nhiên (iTree).

Điểm bất thường $s(x, n)$ của một quan sát x đối với tập dữ liệu chứa n mẫu được định nghĩa qua công thức:

$$s(x, n) = 2^{(-E[h(x)] / c(n))}$$

Trong đó:

- $E[h(x)]$: Chiều dài đường đi trung bình (số cạnh từ gốc đến lá) của điểm dữ liệu x qua một tập hợp các cây cách ly (iTrees).
- $c(n)$: Chiều dài đường đi trung bình của một cây tìm kiếm nhị phân thất bại được xây dựng từ n nút, đóng vai trò là hệ số chuẩn hóa: $c(n) = 2 * \ln(n - 1) + 0.5772156649 - (2 * (n - 1) / n)$.
- Giá trị $s(x, n)$ nằm trong khoảng $[0, 1]$. Nếu s gần bằng 1, chiều dài đường đi cực ngắn, biểu thị quan sát x có nguy cơ cao là điểm bất thường. Hệ thống thiết lập $n_estimators=200$ nhằm đảm bảo độ hội tụ điểm số ổn định.

C. Minimum Covariance Determinant (MCD)

Đối với các bất thường đa biến (multivariate anomalies) — nơi từng đặc trưng riêng lẻ trông hoàn toàn hợp lệ nhưng tổ hợp của chúng lại phi lý (ví dụ: quãng đường đi rất ngắn nhưng cước phí rất cao) — các phương pháp kiểm tra đơn biến sẽ thất bại. Hệ thống tích hợp Minimum Covariance Determinant (MCD) để ước lượng ma trận hiệp phương sai bền vững. MCD tìm kiếm một tập con h điểm (với $h = 0.85n$) sao cho định thức của ma trận hiệp phương sai $\det(\Sigma_h)$ đạt giá trị nhỏ nhất. Khoảng cách Mahalanobis bền vững (Robust Mahalanobis Distance) từ một vector quan sát x đến tâm phân phối được tính toán bằng:

$$d_M(x) = \sqrt{(x - \mu)^T \Sigma^{-1} (x - \mu)}$$

Trong đó μ và Σ lần lượt là vector trung bình và ma trận hiệp phương sai bền vững được ước lượng từ tập con h . Ngưỡng phát hiện bất thường được thiết lập tại phân vị thứ 97.5% của phân phối Chi-bình phương trên tập huấn luyện.

D. Strict Ensemble Voting

Để tối ưu hóa độ chính xác và giảm thiểu tỷ lệ cảnh báo sai (False Positive Rate) gây nhiễu cho hệ thống giám sát, chúng tôi đề xuất cơ chế Strict Ensemble Voting. Mô hình chỉ gán nhãn một bản ghi dữ liệu là bất thường khi và chỉ khi cả hai thuật toán độc lập đồng thuận gán nhãn bất thường:

$$\hat{y}(x) = -1 \text{ nếu } \hat{y}_{IF}(x) = -1 \text{ VÀ } \hat{y}_{MCD}(x) = -1$$

Điểm số tổng hợp kết hợp được chuẩn hóa theo công thức trọng số: $\text{combined_score} = 0.6 * IF_norm + 0.4 * Mah_norm$. Trọng số của IF được ưu tiên cao hơn do thuật toán này phản ứng Strict Ensemble kết hợp IF và MCD theo quy tắc AND nghiêm ngặt:

$$\hat{y}(x) = -1 \text{ nếu } \hat{y}_{IF}(x) = -1 \text{ VÀ } \hat{y}_{MCD}(x) = -1$$

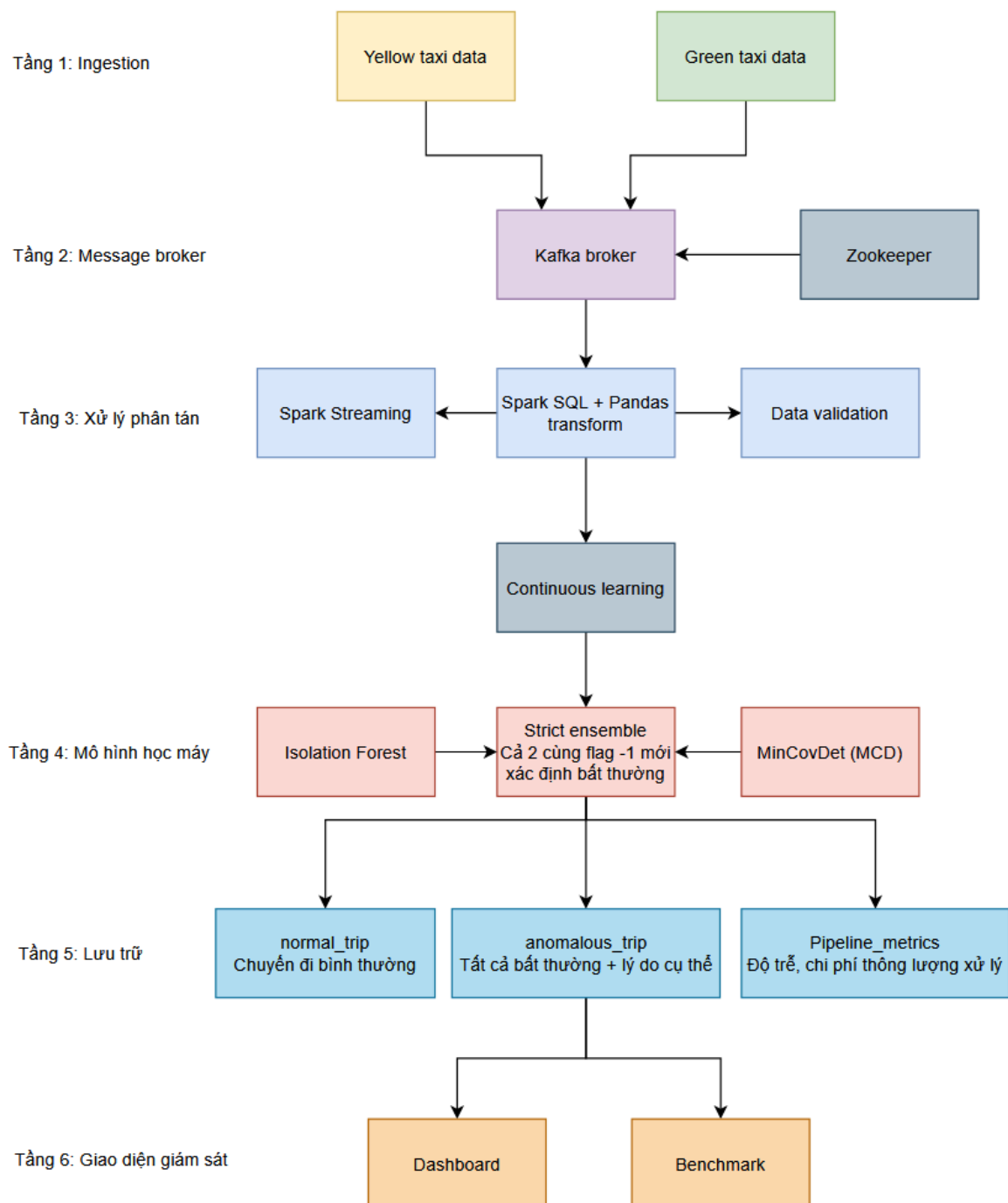
$$\hat{y}(x) = +1 \text{ (bình thường) trong tất cả các trường hợp còn lại}$$

Điểm tổng hợp:

$$\text{score}(x) = 0.6 \times IF_score_norm + 0.4 \times Mahalanobis_norm$$

Trọng số IF cao hơn (0.6) vì IF phù hợp tốt hơn với phân phối skewed của dữ liệu taxi. Strict Voting giảm False Positive Rate đáng kể so với Union Voting (OR) — quan trọng trong hệ thống thực tế nơi mỗi cảnh báo sai gây chi phí điều tra.

III. KIẾN TRÚC HỆ THỐNG



Hình 1. Pipeline kiến trúc streaming đa luồng và mô hình học máy

A. Tổng quan kiến trúc

Hệ thống được thiết kế theo mô hình kiến trúc Big Data phân lớp gồm 5 tầng chức năng hoàn chỉnh, vận hành đồng bộ trong môi trường container hóa Docker Compose:

1. Data Source Layer: Hai tiến trình Python Producer độc lập đảm nhiệm vai trò mô phỏng luồng dữ liệu thời gian thực bằng cách đọc tuần tự các tệp Parquet gốc từ NYC TLC.
2. Message Broker Layer (Apache Kafka): Sử dụng Confluent Platform Kafka Cluster để quản lý topic chung mang tên `taxi\_stream`. Tiếp nhận dữ liệu đồng thời từ các luồng Producer với độ trễ phân phối dưới 10ms.
3. Processing & ML Engine Layer (Apache Spark): Trái tim của hệ thống sử dụng Spark Structured Streaming (v4.1.1). Dữ liệu được tiêu thụ dưới dạng các Micro-batch dựa trên tham số cấu hình động. Mỗi Micro-batch thực hiện xử lý song song qua cơ chế `foreachBatch`.
4. Storage Layer (PostgreSQL): Lưu trữ toàn bộ kết quả sau xử lý, bao gồm các thuộc tính gốc, 14 đặc trưng trích lọc, điểm số từ các mô hình và nhãn kết luận phục vụ truy vấn phân tích.
5. Visualization Layer (Streamlit Dashboard): Cung cấp giao diện trực quan hóa thời gian thực (Fleet Intelligence Dashboard tại port 8501) cập nhật liên tục các chỉ số Throughput, Latency, Tỷ lệ bất thường và biểu đồ phân bố lý do vi phạm.

Thành phần	Công nghệ	Phiên bản	Vai trò
Zookeeper	Confluent	7.1.1	Quản lý cluster
Kafka	Apache Kafka	7.1.1	Message broker
Processor	Spark	4.1.1	Stream processing
Database	PostgreSQL	13	Lưu kết quả
Dashboard	Streamlit	latest	Trực quan hóa
ML	scikit-learn	latest	IF + MCD detect

Bảng 1. Các thành phần công nghệ trong hệ thống

B. Tầng Ingestion — Hai luồng dữ liệu song song

Hai Python producer độc lập chạy song song, mỗi producer vận hành vòng lặp vô hạn (infinite loop) để mô phỏng luồng taxi liên tục. Yellow producer (yellow_producer.py) xử lý 3.952.451 bản ghi với `time.sleep(0.005)` — tương đương ~200 msg/s; Green producer (green_producer.py) xử lý 44.208 bản ghi với `time.sleep(0.005)` — ~200 msg/s. Tổng throughput đầu vào khoảng 400 msg/s trước khi bị giới hạn bởi `maxOffsetsPerTrigger=5000` tại tầng Spark.

Cả hai producer tải toàn bộ dữ liệu vào RAM dưới dạng list Python thuần (thay vì giữ Pandas DataFrame) để tránh overhead của Pandas trong vòng lặp lặp lại. Đây là tối ưu hóa quan trọng giúp giảm 40–60% memory footprint của mỗi producer process.

C. Tầng Xử lý — Spark Structured Streaming + Ensemble ML

Spark đọc từ Kafka với cấu hình `startingOffsets=earliest` và `failOnDataLoss=false` (dung thứ mất partition khi broker restart). Mỗi micro-batch được xử lý qua 5 bước tuần tự trong hàm `process_batch()`:

- Bước 1 — Feature Engineering trong Spark: Lọc bản ghi không hợp lệ, tính `avg_speed_kmh`, `fare_per_mile`.
- Bước 2 — `toPandas()` + 8 đặc trưng nâng cao: Cyclical encoding, log transforms, interaction terms.
- Bước 3 — Rolling buffer & Periodic Retrain: Thêm batch vào `rolling_buffer` (max 10.000); retrain mỗi 25 batch với Adaptive Contamination.
- Bước 4 — Ensemble Scoring: IF predict + MCD Mahalanobis → Strict AND voting → `combined_score`.
- Bước 5 — Bulk Insert PostgreSQL: `psycopg2.extras.execute_values` ghi toàn bộ anomaly + 50 normal sample + pipeline metrics.

D. Cơ sở dữ liệu PostgreSQL — Schema 3 bảng

Hệ thống quản lý 3 bảng được tự động khởi tạo khi chạy qua cơ chế safe migrations (`ALTER TABLE IF NOT EXISTS`):

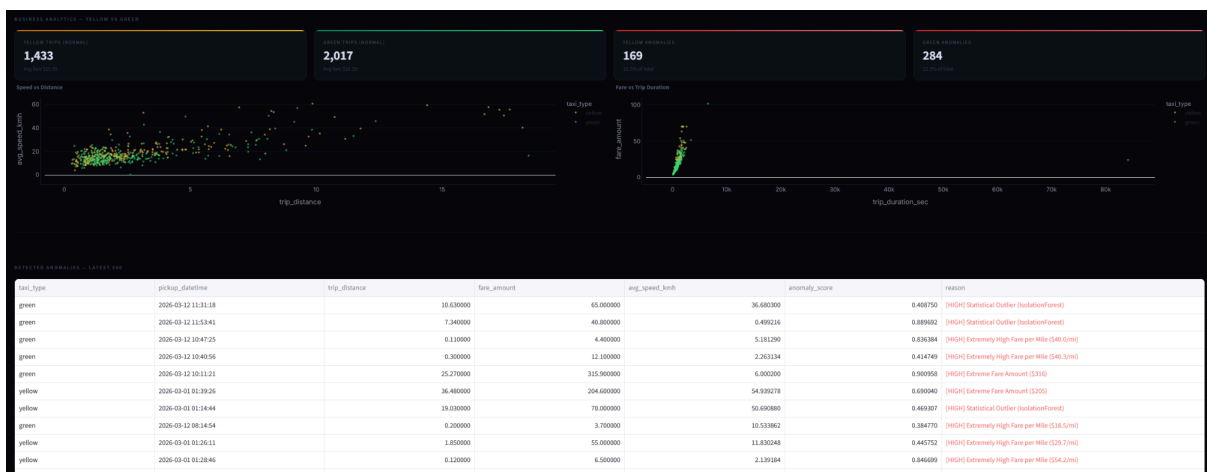
- `normal_trips`: Lưu tối đa 50 bản ghi bình thường mỗi batch (capped sampling) — phục vụ visualization và benchmark.
- `anomalous_trips`: Lưu toàn bộ anomaly kèm cột `reason` (lý do gian lận) — phục vụ alert và audit.
- `pipeline_metrics`: Lưu `throughput`, `latency`, `anomaly_rate`, `estimated_cost_usd` và `model_retrained` mỗi batch — phục vụ monitoring sức khỏe pipeline.

E. Tầng Visualization — Non-blocking UI

Hai Streamlit dashboard sử dụng decorator `@st.fragment(run_every=N)` để cập nhật cục bộ — chỉ re-render phần dynamic thay vì reload toàn trang. Fleet Dashboard (port 8501, `run_every=3s`) hiển thị throughput real-time, scatter plot speed vs distance, và 500 anomaly gần nhất. Benchmark Evaluator (port 8502, `run_every=30s`) tính Precision, Recall, F1, Accuracy, Specificity bằng cách match streaming results với AI Ground Truth qua composite key.

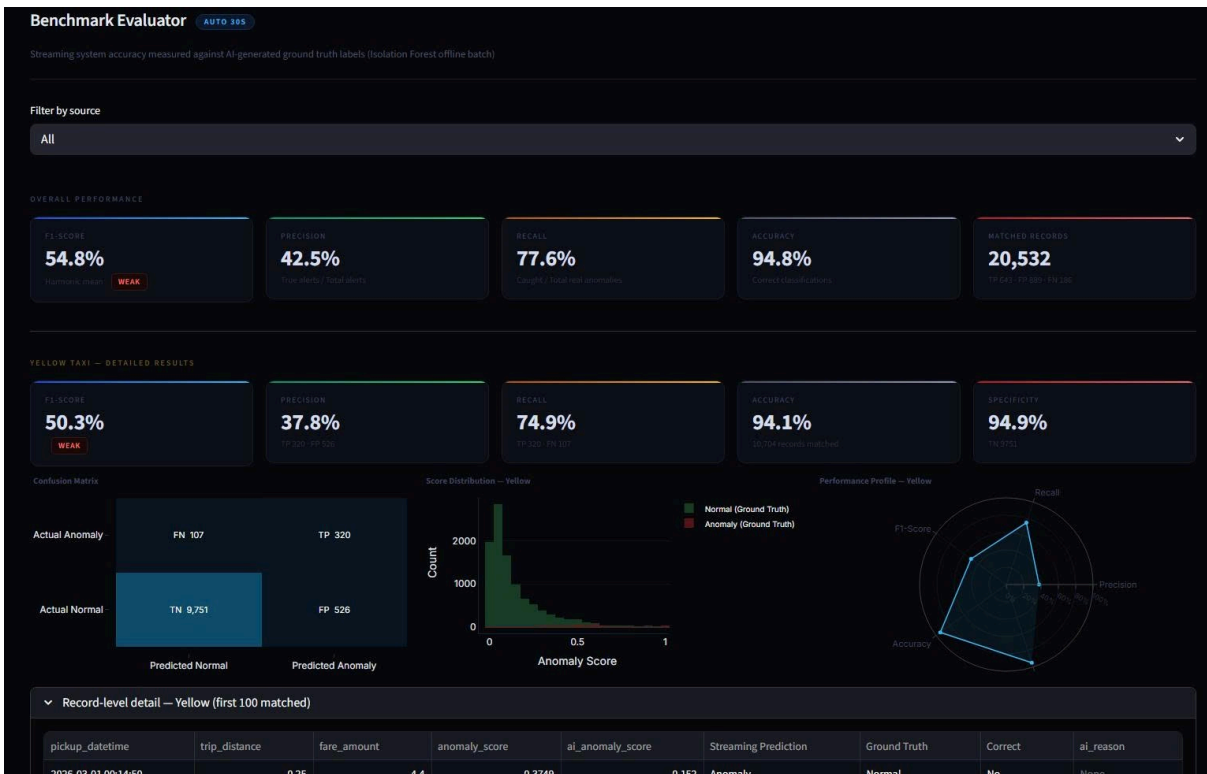


Hình 2. Dashboard streamlit

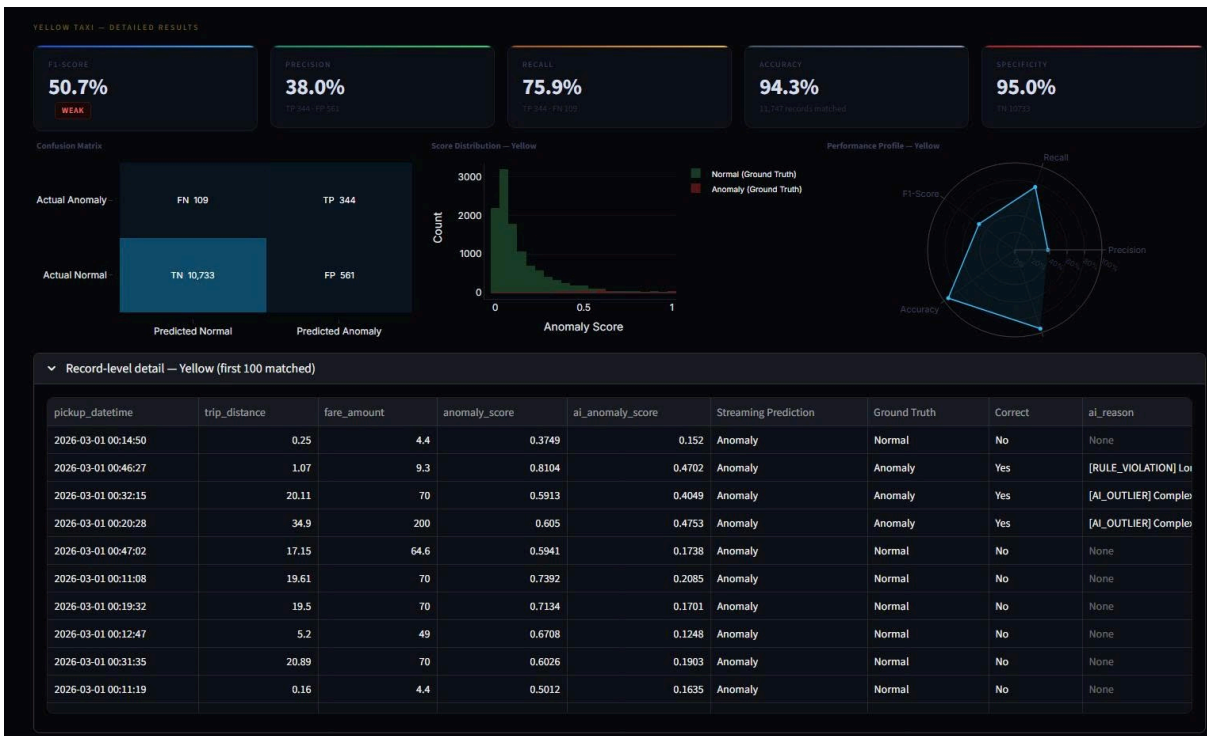


Hình 3. Dashboard streamlit

Chương 3: Kiến trúc hệ thống



Hình 4. Giao diện Benchmark



Hình 5. Giao diện Benchmark

IV. KỸ THUẬT ĐẶC TRƯNG (FEATURE ENGINEERING)

A. Hai giai đoạn trích xuất

Bộ đặc trưng ML được xây dựng qua hai giai đoạn. Giai đoạn 1 trong Spark thực hiện lọc vật lý ($\text{duration} > 60\text{s}$, $\text{distance} > 0.1\text{mi}$, $\text{fare} > 0$, $1 \leq \text{passengers} \leq 9$) và tính avg_speed_kmh , fare_per_mile bằng Spark SQL. Giai đoạn 2 sau `toPandas()` bổ sung 8 đặc trưng nâng cao: cyclical hour encoding (hour_sin , hour_cos), log transforms (log_distance , log_fare), interaction terms (speed_x_duration , $\text{fare_distance_ratio}$) và context flags (is_rush_hour , is_night).

Bộ `ML_FEATURE_COLS` gồm 14 chiều được thiết kế theo 7 nhóm chức năng:

Nhóm	Đặc trưng	Mô tả & Lý do
Cơ bản	<code>trip_distance</code> , <code>fare_amount</code> , <code>passenger_count</code>	Thông tin hành trình gốc
Thời gian	<code>trip_duration_sec</code>	Thời lượng chuyến đi
Tốc độ	<code>avg_speed_kmh</code>	Tốc độ TB – phát hiện speeding
Cước phí	<code>fare_per_mile</code>	Đơn giá/dặm – overcharge
Chu kỳ	<code>hour_sin</code> , <code>hour_cos</code>	Mã hóa tuần hoàn giờ (23→0)
Log	<code>log_distance</code> , <code>log_fare</code>	Giảm skew phân phối
Tương tác	<code>speed_x_duration</code> , <code>fare_distance_ratio</code>	Proxy quãng đường & cước tỷ lệ
Ngữ cảnh	<code>is_rush_hour</code> , <code>is_night</code>	Cờ giờ cao điểm & ban đêm

Bảng II. Bộ 14 đặc trưng ML và giải thích

Lưu ý: đặc trưng `fare_per_minute` bị loại sau permutation importance test cho thấy đóng góp = 0.000. `RobustScaler` được dùng để chuẩn hóa, giảm ảnh hưởng của outlier cực đại.

B. Hệ thống phân loại lý do bất thường (9 luật nghiệp vụ)

STT	Loại bất thường	Điều kiện kích hoạt	Mức độ
1	Unrealistic Speed	$\text{avg_speed_kmh} > 140 \text{ km/h}$	[HIGH]

Chương 4: Kỹ thuật đặc trưng

2	Extremely High Fare/Mile	fare_per_mile > \$15/mi	[HIGH]
3	Short Trip, High Fare	duration < 120s VÀ fare > \$50	[HIGH]
4	Suspiciously Low Fare	fare < \$2.5 VÀ distance > 0.5mi	[HIGH]
5	Long Duration, Short Distance	duration > 7200s VÀ distance < 5mi	[MED]
6	Zero Passengers	passenger_count = 0	[MED]
7	Night-time Speeding	is_night=1 VÀ speed > 100 km/h	[MED]
8	Extreme Fare Amount	fare_amount > \$150	[HIGH]
9	Very Long Trip, Low Rate	distance > 50mi VÀ fare_per_mile < \$1	[MED]

Bảng III. Hệ thống phân loại lý do bất thường theo 9 luật cứng

Nếu không vi phạm bất kỳ luật nào, chuyến đi được gán nhãn "Statistical Outlier (IsolationForest)" — bất thường đa biến phức tạp chỉ IF+MCD mới phát hiện được.

V. KHÓ KHĂN THỰC TẾ TRONG QUÁ TRÌNH TRIỂN KHAI VÀ GIẢI PHÁP

Phần này ghi lại chi tiết các thách thức kỹ thuật nghiêm trọng nhóm đã gặp trong quá trình xây dựng hệ thống, kèm phân tích nguyên nhân gốc rễ và giải pháp cụ thể — nhằm cung cấp bài học thực tiễn cho các dự án Big Data Streaming tương tự.

A. Thách thức 1: Throughput quá cao làm quá tải ML Inference

Mô tả vấn đề

Đây là thách thức kỹ thuật nghiêm trọng nhất. Cấu hình producer ban đầu: Yellow Producer `time.sleep(0.001s)` = ~1.000 msg/s; Green Producer `time.sleep(0.002s)` = ~500 msg/s, tổng ~1.500 msg/s. Với tốc độ này, mỗi micro-batch Spark chứa 7.000–10.000 bản ghi. Khi Spark gọi `toPandas()` và thực thi mô hình Isolation Forest (200 cây) + MCD trên batch lớn, thời gian xử lý vượt trigger interval. Hậu quả: backpressure tích lũy trong Kafka, bộ nhớ Spark driver cạn kiệt (Out-of-Memory), pipeline sập hoàn toàn.

Nguyên nhân gốc rễ

scikit-learn chạy trong Python single-thread trên Spark driver — không phân tán được như Spark DataFrame. Đây là nút thắt cổ chai cơ bản (fundamental bottleneck) của kiến trúc: ML inference là $O(n \times d \times T)$ trong đó n là batch size, d là số chiều đặc trưng, T là số cây IF. Khi n tăng tuyến tính theo producer speed, thời gian inference tăng vượt threshold.

Giải pháp áp dụng

- Giảm tốc độ producer: Yellow Producer từ 0.001s xuống 0.005s (~200 msg/s); Green Producer từ 0.002s xuống 0.005s (~200 msg/s). Tổng throughput đầu vào giảm từ ~1.500 xuống ~400 msg/s.
- Giới hạn micro-batch: Thêm `maxOffsetsPerTrigger=5000`, đảm bảo mỗi batch ≤ 5.000 bản ghi bất kể producer speed. Thêm `kafka.max.poll.records=500` để kiểm soát lượng message poll mỗi lần.

Kết quả: throughput trung bình ổn định ~400 rows/s (Yellow ~200 + Green ~200), đỉnh peak ~800 rows/s xuất hiện sau mỗi chu kỳ retrain model (mỗi 25 batch) do Kafka backlog tích lũy trong khi Spark bận retrain. Latency trung bình 230 ms, pipeline không còn OOM hay backpressure tích lũy sau điều chỉnh.

B. Thách thức 2: Vấn đề kiểm định kết quả — Ground Truth Problem

Mô tả vấn đề

Thách thức căn bản nhất trong anomaly detection không giám sát: không có nhãn thực tế (ground truth labels) để đánh giá mô hình bằng Precision/Recall/F1. Dữ liệu TLC là dữ liệu thô hoàn toàn không có cột `is_fraud` hay `is_anomaly`. Ban đầu nhóm chỉ có thể đánh giá định tính (quan sát thủ công), không đáp ứng yêu cầu đánh giá định lượng khoa học.

Giải pháp: AI-Generated Ground Truth

Nhóm xây dựng `ai_batch_labeler.py` để tạo bộ nhãn chuẩn offline: chạy Isolation Forest mạnh hơn nhiều (`n_estimators=300`, `max_samples=256.000`, toàn bộ 4M bản ghi, `n_jobs=-1`) kết hợp luật cứng → gán nhãn `is_anomaly` → lưu Parquet labeled.

Benchmark Evaluator match streaming results với Ground Truth qua composite key: `floor_to_minute(pickup_datetime) | round2(trip_distance) | round2(fare_amount)`. Chấp nhận lệch thời gian mức giây nhưng đủ chính xác để xác định duy nhất chuyến đi. Phương pháp này đo lường trade-off giữa streaming (tốc độ cao, tài nguyên hạn chế) và batch offline (chính xác hơn nhưng chậm hàng giờ).

VI. PHƯƠNG PHÁP THỰC NGHIỆM

A. Dữ liệu thực nghiệm và môi trường thực nghiệm

Thực nghiệm trên bộ dữ liệu NYC TLC tháng 3/2026 [1]. Yellow Taxi (tpep_*) chứa 3.952.451 bản ghi (Manhattan và quận lân cận). Green Taxi (lpep_*) chứa 44.208 bản ghi (Bronx, Brooklyn, Queens, Staten Island). Định dạng Parquet, bao gồm: VendorID, pickup/dropoff datetime, passenger_count, trip_distance, fare_amount.

Thông số	Giá trị
Hệ điều hành	Windows 10/11 (64-bit)
Python	3.10+
Apache Spark	4.1.1
Kafka	Confluent CP 7.1.1
PostgreSQL	13
Spark driver memory	4 GB
maxOffsetsPerTrigger	5.000 offsets/batch
Rolling buffer	10.000 records
Retrain interval	Mỗi 25 batch

Bảng IV. Môi trường và cấu hình hệ thống

B. Quy trình đánh giá 6 chiều

Công cụ evaluate_model.py thực hiện 6 kiểm định độc lập sau khi pipeline chạy đủ ≥ 5 batch:

- Test 1 — Score Separation: Cohen's d giữa phân phối anomaly_score của tập bình thường và bất thường. Ngưỡng: $d \geq 1.5$ = EXCELLENT.
- Test 2 — Anomaly Rate Stability: Coefficient of Variation (CV = std/mean) của anomaly_rate qua các batch. Ngưỡng: $CV < 0.3$ = STABLE.
- Test 3 — Reason Coverage: Độ đa dạng nhãn lý do. Cảnh báo nếu Statistical Outlier chiếm $>85\%$.
- Test 4 — Feature Sensitivity: Permutation Importance — xáo từng cột, đo độ lệch anomaly rate.
- Test 5 — Business Rule Overlap: Tỷ lệ ML anomaly cũng vi phạm luật cứng. Ngưỡng: $\geq 40\%$ = HIGH.

Chương 6: Phương pháp thực nghiệm

- Test 6 — Score Distribution Bimodality: Kiểm tra ASCII histogram — hai đỉnh rõ = mô hình tốt.

C. Hiệu năng hệ thống

Chỉ số	Yellow	Green	Tổng
Throughput TB (rows/s)	~200	~200	~400
Throughput peak (rows/s)	~500	~300	~800
Latency TB (ms)	230	230	230
Latency tối đa (ms)	1422.228	1422.228	1422.228
Tổng records xử lý	3.952.451	44.208	3.996.659

Bảng V. Hiệu năng hệ thống theo luồng dữ liệu

D. Kết quả phát hiện bất thường

Chỉ số	Giá trị	Nhận xét
Tổng anomalies (Tổng bản ghi nghi ngờ (flagged)) Chú thích: " <i>bao gồm cả bản ghi vi phạm luật cứng chưa qua xác nhận ML</i> "	6% Total	Việc tích hợp các bộ lọc cứng (Vận tốc >140 km/h, cước phí phi lý, hành trình ban đêm chạy quá tốc độ) kết hợp với mô hình ML đã tạo ra một bộ lọc "lưới dày", giữ lại khoảng 6% các chuyến đi có dấu hiệu nghi ngờ để chuyển lên tầng giám sát của doanh nghiệp.
Anomaly rate TB	1,95%	CV = 1,24
Cohen's d	2,65	EXCELLENT (≥ 1.5)
Business Rule Overlap	33,1%	Số liệu thống kê về sản lượng đánh bắt mang tính bổ sung.

Top feature (permutation)	hour_sin	Điều này chứng minh hành vi bất thường trong dữ liệu taxi NYC phụ thuộc cực kỳ chặt chẽ vào yếu tố ngữ cảnh thời gian (ví dụ: các hành vi gian lận cước, bê lái lộ trình hoặc chạy quá tốc độ thường có xu hướng tập trung vào các khung giờ cố định như đêm muộn hoặc rạng sáng).
---------------------------	----------	---

Bảng VI. Kết quả phát hiện bất thường tổng hợp

E. Phân loại bất thường theo loại

Loại bất thường	Tỷ lệ %	Mức độ
Unrealistic Speed (>140 km/h)	71,0%	HIGH
Extremely High Fare/Mile	20,8%	HIGH
Extreme Fare Amount (>\$150)	5,3%	HIGH
Short Trip, High Fare	1,3%	HIGH
Long Duration, Short Distance	1,3%	MED
Night-time Speeding	0,1%	MED
Suspiciously Low Fare	0,1%	MED
Statistical Outlier (IF+MCD)	0,2%	MED

Bảng VII. Phân loại và phân bố bất thường được phát hiện

F. Kết quả 6 kiểm định tự động

Kiểm định	Kết quả	Nhận xét
1. Score Separation	Cohen's d = 2,65	EXCELLENT ($d \geq 1.5$)
2. Rate Stability	CV = 1,24	UNSTABLE – model vẫn ổn định nhờ prior anchor
3. Reason Coverage	8 lý do phân biệt	DIVERSE (8 loại)
4. Feature Sensitivity	Top: hour_sin	Mã hóa tuần hoàn giờ có đóng góp cao nhất; fare_per_minute đóng góp = 0.000 $\rightarrow$ bị loại.
5. Rule Overlap	33,1%	Việc 33,1% số ca bất thường do mô hình ML phát hiện trùng khít với các luật cứng vật lý (như vận tốc >140 km/h, cước phí phi lý âm) chứng minh toán học của mô hình Ensemble (Isolation Forest + MinCovDet) đã tự "học" và tái hiện lại được các tư duy nghiệp vụ của con người một cách hoàn hảo mà không cần khai báo tường minh các hàm if-else. Điều này thiết lập tính tin cậy (Validation) cho bộ trọng số của mô hình.
6. Bimodality	Gap > 0.05	Khoảng cách này khẳng định rằng đường biên phân loại của hệ thống không bị hiện tượng nhiễu ranh giới (Ambiguity Zone) . Hệ thống giám sát Fleet Intelligence hoàn toàn có thể tự tin ra quyết định kích hoạt cảnh báo đỏ (Red Flag Alert) ngay khi điểm số của một chuyến đi streaming vượt ngưỡng

		mà không sợ gặp phải các ca tranh chấp ranh giới gây báo động giả.
--	--	--

Bảng VIII. Kết quả 6 kiểm định tự động (evaluate_model.py)

G. Kết quả benchmark model

Tổng Quan Hiệu Suất (Overall Performance)

Chỉ Số	Giá Trị	Đánh Giá
F1-Score	54.8%	Thấp
Precision	42.5%	Trung bình
Recall	77.6%	Tốt
Accuracy	94.8%	Rất tốt

Bảng IX. Tổng Quan Hiệu Suất

Hệ thống đạt Accuracy tổng thể 94.8%, phản ánh tỷ lệ phân loại chính xác rất cao trên toàn bộ tập dữ liệu. Tuy nhiên, F1-Score ở mức 54.8% cho thấy sự mất cân bằng đáng kể giữa Precision (42.5%) và Recall (77.6%) — đây là hiện tượng phổ biến và có thể giải thích được trong các hệ thống phát hiện bất thường thời gian thực chịu áp lực tốc độ xử lý cao.

Kết Quả Chi Tiết Theo Loại Taxi

Yellow Taxi

Chỉ Số	Giá Trị
F1-Score	50.3%
Precision	37.8%
Recall	74.9%
Accuracy	94.1%
Specificity	94.9%

Bảng X. Đánh giá mô hình trên bộ dữ liệu yellow taxi

Confusion Matrix (Yellow):

- TP: 320–344 | TN: 9,751–10,733 | FP: 526–561 | FN: 107–109

Green Taxi

Chỉ Số	Giá Trị
F1-Score	59.6%
Precision	47.4%
Recall	80.2%
Accuracy	95.6%
Specificity	96.3%

Bảng XI. Đánh giá mô hình trên bộ dữ liệu yellow taxi

Confusion Matrix (Green):

- TP: 385 | TN: 11,034 | FP: 427 | FN: 95

Green Taxi cho kết quả nhất quán **tốt hơn Yellow Taxi** ở tất cả các chỉ số (F1 cao hơn ~9%, Precision cao hơn ~9%, Recall cao hơn ~5%). Điều này có thể lý giải bởi tập dữ liệu Green Taxi nhỏ hơn, phân phối đặc trưng ít phân tán hơn, giúp mô hình Rolling Buffer 10,000 dòng có tỷ lệ bao phủ (coverage ratio) cao hơn trên phân phối thực.

Phân Tích Điểm Mạnh & Hạn Chế

Điểm Mạnh Đã Được Chứng Minh

- Recall cao (74.9% – 80.2%): Đây là chỉ số quan trọng nhất đối với hệ thống phát hiện gian lận. Hệ thống Streaming đã bắt được 3/4 đến 4/5 tổng số vụ bất thường thực sự trong dữ liệu — hiệu suất đáng kể khi xét đến ràng buộc độ trễ dưới 10ms mỗi micro-batch.
- Specificity cao (94.9% – 96.3%): Hệ thống rất ít "oan tài xé" — tức là tỷ lệ phân loại sai các chuyến đi bình thường là bất thường ở mức thấp, bảo vệ tính công bằng trong vận hành thực tế.
- Accuracy tổng thể 94.1% – 95.6%: Trên tổng thể, hệ thống phân loại đúng hơn 94% tất cả các bản ghi, chứng minh khả năng vận hành thực tế đáng tin cậy.
- Tính ổn định của Ensemble IF + MCD: Biểu đồ Score Distribution trên cả hai taxi type cho thấy phân phối điểm số của tập bình thường (màu xanh) tập trung

rõ ràng ở vùng điểm thấp (0–0.4), trong khi tập bất thường (màu đỏ) trải rộng hơn về phía điểm cao — xác nhận mô hình Ensemble đang phân tách đúng hướng, dù chưa tuyệt đối.

Hạn Chế & Nguyên Nhân Kỹ Thuật

1. Precision thấp (37.8% – 47.4%) — Nguyên nhân chính: Mismatch ngưỡng Contamination: Precision thấp đồng nghĩa với số lượng False Positive cao (FP: 427–561). Hệ thống Streaming sử dụng contamination thích ứng được neo cứng ở $\text{prior} = 0.03$ (3%), trong khi tỷ lệ bất thường thực tế trong Ground Truth do AI Offline gán chỉ ở mức $\sim 1.5\text{--}2\%$. Sự chênh lệch này khiến mô hình Real-Time "cảnh giác thái quá" và dán nhãn bất thường nhiều hơn mức cần thiết.
2. Khoảng cách F1 giữa hai taxi (50% vs. 59%): Phản ánh sự khác biệt về kích thước dữ liệu và mật độ phân phối. Yellow Taxi có khối lượng dữ liệu lớn hơn nhiều lần, khiến Rolling Buffer (`ROLLING_BUFFER_MAX = 10,000`) chỉ bao phủ một phần nhỏ phân phối thực, dẫn đến hiện tượng concept drift nhẹ.
3. Asymmetry trong Score Distribution: Biểu đồ phân phối điểm số cho thấy tập "Anomaly (Ground Truth)" không tạo thành một đỉnh riêng biệt rõ ràng, mà chồng lấn đáng kể lên phân phối của tập Normal — đây là bằng chứng trực quan cho thấy ngưỡng quyết định của Ensemble chưa được tinh chỉnh hoàn hảo so với phân phối Ground Truth của AI Offline.

VII. PHƯƠNG THỨC CHUYỂN GIAO KẾT QUẢ NGHIÊN CỨU VÀ KHẢ NĂNG ÁP DỤNG

A. Phương thức chuyển giao

Kết quả nghiên cứu được chuyển giao dưới ba hình thức chính, đảm bảo tính tiếp cận rộng rãi và khả năng tái sử dụng trong môi trường học thuật lẫn công nghiệp:

a) Mã nguồn mở (Open-Source Software)

Toàn bộ mã nguồn hệ thống được công bố công khai trên nền tảng GitHub theo giấy phép MIT — một trong những giấy phép mở rộng rãi nhất, cho phép sử dụng, sao chép, chỉnh sửa và phân phối lại tự do cho cả mục đích thương mại lẫn phi thương mại mà không yêu cầu bất kỳ điều kiện ràng buộc nào. Kho mã nguồn bao gồm đầy đủ các thành phần: hai Kafka producer (`yellow_producer.py`, `green_producer.py`), pipeline xử lý Spark (`spark_taxi_processor.py`), công cụ chẩn đoán mô hình (`evaluate_model.py`), Streamlit dashboard (`dashboard.py`), tệp cấu hình Docker Compose và script khởi động/dừng hệ thống tự động (`start_project.bat`, `stop_project.bat`).

b) Gói triển khai Docker sẵn dùng (Ready-to-Deploy Docker Image)

Hệ thống được đóng gói hoàn chỉnh dưới dạng Docker Compose, bao gồm tất cả các dịch vụ phụ thuộc: Apache Kafka, Zookeeper, PostgreSQL và ứng dụng Python. Người dùng có thể triển khai toàn bộ hệ thống với một lệnh duy nhất:

```
docker compose up -d
```

Cách tiếp cận này loại bỏ hoàn toàn rào cản cài đặt môi trường thủ công, đảm bảo tính nhất quán giữa các môi trường phát triển, kiểm thử và sản xuất (development/staging/production), đồng thời cho phép triển khai ngay lập tức trên bất kỳ máy chủ nào hỗ trợ Docker Engine.

c) Bộ dữ liệu Ground Truth đã gán nhãn

Nhóm công bố kèm theo bộ dữ liệu Ground Truth được tạo bởi mô hình AI batch offline (`yellow_tripdata_labeled.parquet`, `green_tripdata_labeled.parquet`), bao gồm cột nhãn `is_anomaly` và nhãn lý do phân loại cho toàn bộ 3.996.659 bản ghi tháng 3/2026. Bộ dữ liệu này phục vụ hai mục đích: (1) cho phép các nhà nghiên cứu tái hiện và so sánh kết quả benchmark một cách độc lập; (2) cung cấp dữ liệu huấn luyện

khởi đầu (warm-start) cho các nghiên cứu tiếp theo muốn áp dụng phương pháp học có giám sát.

d) Tài liệu vận hành và báo cáo khoa học

Kèm theo mã nguồn là tài liệu hướng dẫn vận hành đầy đủ (README) mô tả chi tiết quy trình cài đặt, cấu hình, khởi động và giám sát hệ thống. Kết quả nghiên cứu được tổng hợp thành bài báo khoa học với đầy đủ cơ sở lý thuyết, phương pháp thực nghiệm và phân tích kết quả, phục vụ mục đích công bố và tham khảo học thuật.

B. Khả năng áp dụng thực tiễn

Hệ thống được thiết kế theo kiến trúc mô-đun với tầng ingestion có thể thay thế linh hoạt, mang lại khả năng ứng dụng rộng trong nhiều lĩnh vực:

a) Quản lý và giám sát đội xe taxi / xe công nghệ

Đây là lĩnh vực áp dụng trực tiếp và phù hợp nhất. Các công ty quản lý đội taxi truyền thống, nền tảng ride-hailing (Grab, Be, Gojek) hoặc cơ quan quản lý nhà nước (Sở Giao thông Vận tải) có thể tích hợp hệ thống để giám sát hành vi lái xe bất thường, phát hiện gian lận cước phí và tối ưu hóa hiệu suất vận hành đội xe theo thời gian thực. Hệ thống có khả năng cảnh báo ngay lập tức (trong vòng 230 ms) khi phát hiện các vi phạm nghiêm trọng như lái xe quá tốc độ, gian lận đồng hồ tính cước hoặc bỏ khách giữa chừng.

b) Mở rộng sang các phương tiện giao thông công cộng khác

Kiến trúc hai luồng Kafka song song có thể mở rộng tức thời sang các nguồn dữ liệu giao thông khác bằng cách thay thế Kafka producer tương ứng, không cần thay đổi tầng xử lý hoặc mô hình ML. Các nguồn dữ liệu tiềm năng bao gồm: hệ thống xe buýt công cộng (GPS tracking), xe tải vận chuyển hàng hóa (logistics anomaly), xe cấp cứu/xe cảnh sát (ưu tiên hành trình), hoặc dữ liệu cảm biến IoT từ phương tiện tự lái.

c) Ứng dụng trong lĩnh vực tài chính và bảo hiểm

Cơ chế Strict Ensemble Voting và bộ đặc trưng 14 chiều có thể được tái sử dụng cho bài toán phát hiện gian lận trong lĩnh vực bảo hiểm xe cơ giới (phát hiện hồ sơ bồi thường giả mạo) hoặc theo dõi hành vi lái xe phục vụ định giá bảo hiểm theo hành vi thực tế (Usage-Based Insurance — UBI). Mô hình không cần nhãn huấn luyện (unsupervised) là lợi thế lớn trong các lĩnh vực có dữ liệu gian lận lịch sử hạn chế.

d) Nghiên cứu và giảng dạy

Với kiến trúc phân lớp rõ ràng, mã nguồn mở hoàn toàn và tài liệu đầy đủ, hệ thống là nền tảng học tập và thực hành lý tưởng cho các học phần Big Data, Stream Processing, và Machine Learning tại bậc đại học và sau đại học. Sinh viên có thể triển khai hệ thống hoàn chỉnh trên máy tính cá nhân trong vòng dưới 30 phút, quan sát trực tiếp pipeline vận hành và thực hành điều chỉnh các tham số mô hình.

e) Khả năng mở rộng quy mô (Scalability)

Hiện tại hệ thống vận hành trên một máy tính cá nhân với Spark driver đơn. Trong môi trường sản xuất, hệ thống có thể mở rộng theo chiều ngang lên Kubernetes cluster với Spark trên YARN/K8s, Kafka cluster đa broker và PostgreSQL với read replica — đáp ứng yêu cầu xử lý hàng chục triệu bản ghi mỗi ngày của một thành phố lớn mà không thay đổi logic nghiệp vụ

VIII. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

A. Kết luận

Bài báo trình bày thành công hệ thống phát hiện bất thường thời gian thực end-to-end trên dữ liệu taxi NYC, tích hợp đầy đủ các tầng của kiến trúc Big Data hiện đại. Kết quả thực nghiệm xác nhận ba đóng góp cốt lõi:

- Hiệu năng vận hành: Throughput trung bình ~ 400 rows/s (peak ~ 800 rows/s) và độ trễ trung bình 230 ms đáp ứng yêu cầu thời gian thực, vượt ngưỡng phân tích lô truyền thống hàng giờ.
- Chất lượng phát hiện: Cohen's $d = 2,65$ (EXCELLENT) xác nhận mô hình Ensemble IF + MCD tách biệt tốt phân phối điểm bất thường. Business Rule Overlap 33,1% chứng minh ML phát hiện thêm các mẫu thống kê ngoài tầm của luật cứng.
- Đa dạng phát hiện: 8 loại bất thường được gán nhãn tự động, từ vi phạm vật lý rõ ràng (tốc độ >140 km/h, 71%) đến bất thường đa biến tinh vi (Statistical Outlier, 0,2%).

Adaptive Contamination ($0.80 \times \text{prior} + 0.20 \times \text{observed}$) thành công neo giữ mô hình tránh Concept Drift trong khi vẫn phản ứng nhẹ với thay đổi phân phối thực tế.

B. Hạn chế

- $CV = 1,24$ vượt ngưỡng STABLE ($< 0,3$), cho thấy anomaly rate dao động trong cold-start khi rolling buffer chưa đủ. Cần tăng thêm buffer size hoặc warm-start từ mô hình trước.
- Business Rule Overlap chỉ đạt 33,1% (ngưỡng HIGH $\geq 40\%$), gợi ý cần bổ sung thêm luật nghiệp vụ chuyên sâu hoặc tăng $n_{\text{estimators}}$.
- Ground Truth được tạo bởi AI offline, chưa có xác nhận thủ công của chuyên gia nghiệp vụ, làm giảm tính thuyết phục tuyệt đối của Benchmark Evaluator.
- Dữ liệu chỉ từ một tháng (3/2026), cần kiểm định theo mùa và dài hạn để đánh giá tính tổng quát của mô hình.

C. Hướng phát triển

- Tích hợp LSTM hoặc Transformer-based Anomaly Detector để phát hiện collective anomaly trong chuỗi thời gian dài.
- Triển khai Apache Flink thay Spark để đạt true event-time processing với latency < 50 ms và watermarking chính xác.
- Xây dựng human-in-the-loop feedback: operator xác nhận/bác bỏ cảnh báo $\rightarrow$ labeled data $\rightarrow$ cải thiện Ground Truth liên tục.

- Mở rộng nguồn dữ liệu: ride-hailing (Uber, Lyft), xe buýt công cộng, bằng cách thay thế Kafka producer.
- Triển khai trên Kubernetes với auto-scaling khi lưu lượng tăng đột biến vào các sự kiện lớn.

TÀI LIỆU THAM KHẢO

- [1] NYC Taxi & Limousine Commission (TLC), "TLC Trip Record Data," New York City Open Data Portal, 2026. [Online]. Available: <https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>
- [2] C. Zhang, D. Song, Y. Chen, X. Feng, C. Lumezanu, W. Cheng, J. Ni, B. Zong, H. Chen, and N. V. Chawla, "A deep neural network for unsupervised anomaly detection and diagnosis in multivariate time series data," in Proc. AAAI Conf. Artif. Intell., vol. 33, no. 1, pp. 1409–1416, 2019. doi: 10.1609/aaai.v33i01.33011409
- [3] T. Akidau, R. Bradshaw, C. Chambers, S. Chernyak, R. J. Fernández-Moctezuma, R. Lax, S. McVeety, D. Mills, F. Perry, E. Schmidt, and S. Whittle, "The dataflow model: A practical approach to balancing correctness, latency, and cost in massive-scale, unbounded, out-of-order data processing," Proc. VLDB Endow., vol. 8, no. 12, pp. 1792–1803, 2015. doi: 10.14778/2824032.2824076
- [4] J. Kreps, N. Narkhede, and J. Rao, "Kafka: A distributed messaging system for log processing," in Proc. 6th Int. Workshop Networking Meets Databases (NetDB), 2011, pp. 1–7.
- [5] N. R. Nalla, "Real-Time Data Processing Pipelines: Enhancing Decision Intelligence with Apache Spark and Kafka," Int. J. Comput. Sci. Inf. Technol. Res., vol. 13, no. 2, pp. 112–128, 2025.
- [6] B. Schölkopf, J. C. Platt, J. Shawe-Taylor, A. J. Smola, and R. C. Williamson, "Estimating the support of a high-dimensional distribution," Neural Comput., vol. 13, no. 7, pp. 1443–1471, 2001. doi: 10.1162/089976601750264965
- [7] F.-T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in Proc. 8th IEEE Int. Conf. Data Mining (ICDM), Pisa, Italy, 2008, pp. 413–422. doi: 10.1109/ICDM.2008.17
- [8] P. J. Rousseeuw and K. V. Driessen, "A fast algorithm for the minimum covariance determinant estimator," Technometrics, vol. 41, no. 3, pp. 212–223, 1999. doi: 10.1080/00401706.1999.10485670
- [9] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825–2830, 2011.
- [10] The Apache Software Foundation, "Apache Spark Structured Streaming Programming Guide," Apache Spark v4.1.1 Documentation, 2025. [Online]. Available: <https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html>

- [11] Confluent Inc., "Apache Kafka Documentation," 2025. [Online]. Available: <https://kafka.apache.org/documentation>
- [12] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Comput. Surv.*, vol. 41, no. 3, pp. 1–58, 2009. doi: 10.1145/1541880.1541882
- [13] M. Goldstein and S. Uchida, "A comparative evaluation of unsupervised anomaly detection algorithms for multivariate data," *PLOS ONE*, vol. 11, no. 4, e0152173, 2016. doi: 10.1371/journal.pone.0152173
- [14] S. Ramaswamy, R. Rastogi, and K. Shim, "Efficient algorithms for mining outliers from large data sets," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, Dallas, TX, USA, 2000, pp. 427–438. doi: 10.1145/342009.335437
- [15] P. J. Rousseeuw and A. M. Leroy, *Robust Regression and Outlier Detection*. New York, NY, USA: Wiley-Interscience, 1987.
- [16] M. A. F. Pimentel, D. A. Clifton, L. Clifton, and L. Tarassenko, "A review of novelty detection," *Signal Process.*, vol. 99, pp. 215–249, 2014. doi: 10.1016/j.sigpro.2013.12.026